Topic 4: Doubly and Circular Linked Lists

4.1 Doubly Linked List

- Each node has two pointers: next (forward) and prev (backward)
- Allows traversal in both directions -- bidirectional iteration
- O(1) deletion when you have a pointer to the node itself (no need to find previous node)
- More memory per node than singly linked list

```
struct DNode {
    int    data;
    DNode* next;
    DNode* prev;
    DNode(int d) : data(d), next(nullptr), prev(nullptr) {}
};
```

4.2 DLL Insert at Front and End

- insertFront: new node's next = head; if head exists set head->prev = new node; head = new node
- insertEnd: traverse to tail; tail->next = new node; new node->prev = tail; tail = new node
- Maintaining a tail pointer makes insertEnd O(1) instead of O(n)

```
void insertFront(int d) {
    DNode* n = new DNode(d);
    n->next = head;
    if (head) head->prev = n;
    head = n;
}
```

4.3 DLL Delete a Node

- If node->prev exists: node->prev->next = node->next
- If node->next exists: node->next->prev = node->prev
- If deleting head: update head = head->next
- delete node

4.4 Circular Linked List

- The last node's next points back to the head instead of nullptr
- Useful for round-robin scheduling, music playlist looping

- Singly circular: only next pointers form a cycle
- Doubly circular: both next and prev pointers form cycles
- Traversal must check for cycle: stop when current->next == head

```
// Insert in singly circular list
void insertEnd(int d) {
    Node* n = new Node(d);
    if (!head) { head = n; n->next = head; return; }
    Node* cur = head;
    while (cur->next != head) cur = cur->next;
    cur->next = n;
    n->next   = head;
}
```
